

Lab 4

name: WangShiyu nuist_id: 202383910020 setu_id: W20109917

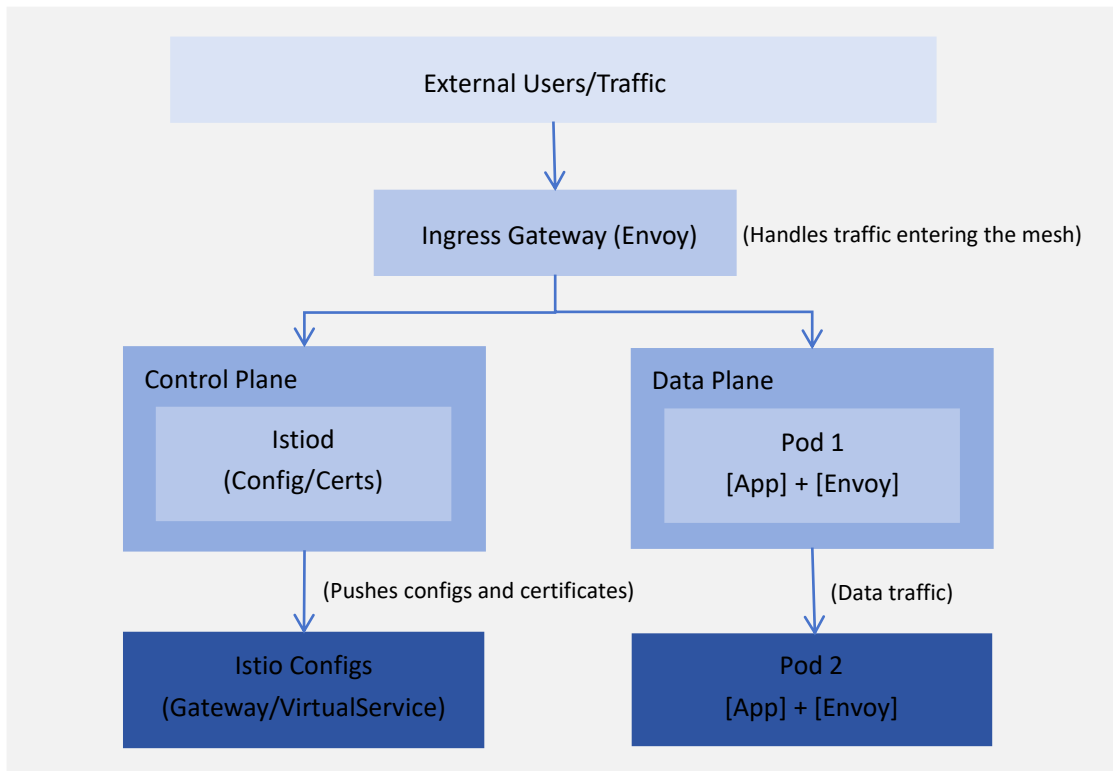
1. Experiment Purpose

- 1) **Understand Core Concepts:** Gain a deep understanding of the basic architecture of Istio Service Mesh, including the collaboration mechanism between the control plane and the data plane.
- 2) **Master Installation and Deployment:** Learn how to download, install, and configure Istio on a Kubernetes cluster, and understand the usage of the `istioctl` command-line tool.
- 3) **Practice Microservices Deployment:** Deploy a typical microservice application and comprehend the principle and verification method of Istio's Sidecar auto-injection.
- 4) **Configure Traffic Management:** Master the method of using Istio Gateway and VirtualService to expose services to external traffic.
- 5) **Experience Observability:** Deploy and use the Kiali dashboard to intuitively feel the service topology, traffic monitoring, and health status check capabilities provided by the Service Mesh.

2. Experiment Process

Istio is an open-source service mesh that provides a uniform way to connect, secure.

The following diagram illustrates the core components of Istio and their positions within a Kubernetes cluster.



1) Step 1: Environment Verification

Confirm that the Kubernetes cluster is functioning correctly, serving as the foundation for all subsequent operations.

```
PS C:\Users\WangShiyu> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:54880
CoreDNS is running at https://127.0.0.1:54880/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\WangShiyu> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane   122m   v1.35.1
```

- The kubectl cluster-info output confirmed the running addresses of the control plane and the core DNS service, proving that the cluster control plane had started successfully.
- kubectl get nodes showed the node minikube with a status of Ready and role control-plane, indicating the cluster was ready.

2) Step 2: Download and Install Istio

Download the Istio distribution and add its command-line tool istioctl to the system PATH.

(1) Download and extract the Istio installation package

Set the Istio version variable, download the Windows version of Istio using an accelerated mirror and unzip the package. (Due to network constraints, the ghfast.top acceleration mirror was used to download from GitHub.)

```
PS C:\Users\WangShiyu> $ISTIO_VERSION = "1.28.0"
PS C:\Users\WangShiyu> Invoke-WebRequest -Uri "https://ghfast.top/https://github.com/istio/istio/releases/download/$ISTIO_VERSION/istio-$ISTIO_VERSION-win.zip" -OutFile "istio.zip"
PS C:\Users\WangShiyu> Expand-Archive -Path "istio.zip" -DestinationPath "."
```

After extraction, the directory structure includes:

- bin/: Contains the istioctl.exe command-line tool.
- samples/: Contains sample applications like Bookinfo.
- manifests/: Contains YAML configuration files for installation.

(2) Add istioctl to PATH

Navigate to the unzipped directory and temporarily add the bin directory in the current directory to the user's PATH environment variable.

```
PS C:\Users\WangShiyu> cd istio-1.28.0
PS C:\Users\WangShiyu\istio-1.28.0> $env:PATH += ";$PWD\bin"
```

(3) Verify Installation

Check the istioctl version

```
PS C:\Users\WangShiyu\istio-1.28.0> istioctl version
2026-05-12T05:40:31.490756Z    warn    default revision watcher not synced, falling back to "default"
Istio is not present in the cluster: no running Istio pods in namespace "istio-system"
2026-05-12T05:40:31.505168Z    warn    default revision watcher not synced, falling back to "default"
client version: 1.28.0
```

- client version: 1.28.0 indicates that the istioctl client was successfully installed and recognized its version.
- The warning Istio is not present in the cluster is normal, as Istio has not yet been installed to the Kubernetes cluster. This command verified the availability of the client tool.

3) Step 3: Install Istio to the Cluster

Install Istio into the Kubernetes cluster using the demo profile and enable the Sidecar

auto-injection feature.

(1) Create a Custom Configuration File

Create an IstioOperator configuration file, specifying the use of a domestic mirror source.

```
PS C:\Users\WangShiyu\istio-1.28.0> @"
>> apiVersion: install.istio.io/v1alpha1
>> kind: IstioOperator
>> spec:
>>   profile: demo
>>   hub: docker.1ms.run/istio
>> "@ | Out-File -FilePath "istio-config.yaml" -Encoding utf8
PS C:\Users\WangShiyu\istio-1.28.0> Get-Content istio-config.yaml
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: demo
  hub: docker.1ms.run/istio
```

- profile: demo: Uses Istio's demo profile, which includes Istio's core components and is suitable for learning and testing.
- hub: docker.1ms.run/istio: A critical configuration. Specifies pulling all required Istio images from the domestic mirror repository docker.1ms.run, solving the problem of slow or failed pulls directly from Docker Hub.

(2) Execute the Installation Command

Install using the custom configuration file.

```
PS C:\Users\WangShiyu\istio-1.28.0> istioctl install -f istio-config.yaml -y
```



```
WARNING: Istio 1.26.0 may be out of support (EOL) already: see https://istio.io/latest/docs/releases/supported-releases/
for supported releases
✔ Istio core installed 🚀
✔ Istiod installed 🌸
✔ Ingress gateways installed 🚢
✔ Egress gateways installed 🚢
✔ Installation complete 🎉
```

The installation process completed successfully, indicating that Istio's core components, including Istiod, Ingress Gateway, Egress Gateway, were deployed to the istio-system namespace in the cluster.

(3) Verify Installation Result

Check the status of Pods in the istio-system namespace.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods -n istio-system
```

NAME	READY	STATUS	RESTARTS	AGE
istio-egressgateway-5484c95fdb-4dbdr	1/1	Running	0	3m50s
istio-ingressgateway-74c878c76-82gjr	1/1	Running	0	3m50s
istiod-599bd565bd-nhg8g	1/1	Running	0	8m26s

All Pods for Istio's core components are in a Running state, with a READY status of 1/1, indicating that the containers have started successfully and are ready.

(4) Enable Sidecar Auto-Injection

Label the default namespace to enable auto-injection, and verify the label was set successfully.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl label namespace default istio-injection=enabled
namespace/default labeled
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get namespace -L istio-injection
NAME          STATUS   AGE    ISTIO-INJECTION
default       Active   9h     enabled
istio-system  Active   6h56m
kube-node-lease Active   9h
kube-public   Active   9h
kube-system   Active   9h
```

When a namespace is labeled `istio-injection=enabled`, Istio's admission controller listens for Pod creation requests in that namespace. Before a Pod is created, it automatically injects an Envoy Sidecar container into the Pod's definition. This allows all traffic to and from containers in that Pod to be intercepted and handled by the Envoy proxy, providing all Istio's benefits without modifying the application code.

4) Step 4: Install Gateway API CRDs

Install the Kubernetes Gateway API Custom Resource Definitions (CRDs), which are required for Istio to support the standard Kubernetes Gateway API.

(1) Create Gateway API CRD Configuration File

Create a YAML file containing the definitions for Gateway, HTTPRoute, and GatewayClass CRDs.

```
c:\Users\wanghuiy\latisio>ls -la g'
apiVersion: apitemplate.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: gateway.gateway.networking.k8s.io
  annotations:
    api-approved.kubernetes.io: https://github.com/kubernetes-sigs/gateway-api/pull/1923
spec:
  group: gateway.networking.k8s.io
  names:
    kind: Gateway
    listKind: GatewayList
    plural: gateways
    singular: gateway
    categories:
      - gateway-api
  scope: Namespaced
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                gatewayClassName:
                  type: string
                  listSetter:
                    type: array
                    items:
                      type: object
                      required:
                        name
                      properties:
                        port:
                          type: integer
                        protocol:
                            type: string
                        allowedRoutes:
                          type: object
                          properties:
                            namespaces:
                              type: object
                              properties:
                                from:
                                  type: string
            status:
              type: object
              properties:
                conditions:
                  type: array
                  items:
                    type: object
                    properties:
                      name:
                        type: string
                      port:
                        type: integer
status:
  type: object
  x-kubernetes-preserve-unknown-fields: true
---
apiVersion: apitemplate.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: gatewayclasses.gateway.networking.k8s.io
  annotations:
    api-approved.kubernetes.io: https://github.com/kubernetes-sigs/gateway-api/pull/1923
spec:
  group: gateway.networking.k8s.io
  names:
    kind: GatewayClass
    listKind: GatewayClassList
    plural: gatewayclasses
    singular: gatewayclass
    categories:
      - gateway-api
  scope: Cluster
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                controllerName:
                  type: string
```

Due to restricted network environments, this step can create the CRD definitions for the Kubernetes Gateway API instead of using `kubectl kustomize` from the official documentation.

(2) Apply Gateway API CRDs

Apply the created CRD configuration file.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f gateway-api-crd.yaml
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
```

The output confirms that the three key CRDs, including Gateway, HTTPRoute, GatewayClass, have been successfully created in the Kubernetes cluster.

(3) Verify CRD Installation

Check if the Gateway API related CRDs are installed.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get crd | Select-String "gateway"
gatewayclasses.gateway.networking.k8s.io    2026-05-12T15:31:13Z
gateways.gateway.networking.k8s.io          2026-05-12T15:31:13Z
gateways.networking.istio.io                2026-05-12T12:59:25Z
httproutes.gateway.networking.k8s.io        2026-05-12T15:31:13Z
```

The result shows four CRDs with "gateway" in their name. The three ending in gateway.networking.k8s.io are the standard Kubernetes Gateway API CRDs we just installed. The gateways.networking.istio.io is Istio's own CRD, which was installed automatically during the Istio installation.

(4) Create Istio GatewayClass

Create a GatewayClass named 'istio', which is required for the Kubernetes Gateway API to work with Istio.

```
PS C:\Users\WangShiyu\istio-1.28.0> @"
>> apiVersion: gateway.networking.k8s.io/v1
>> kind: GatewayClass
>> metadata:
>>   name: istio
>> spec:
>>   controllerName: istio.io/gateway-controller
>> "@ | Out-File -FilePath "istio-gatewayclass.yaml" -Encoding utf8
PS C:\Users\WangShiyu\istio-1.28.0>
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f istio-gatewayclass.yaml
gatewayclass.gateway.networking.k8s.io/istio created
```

(5) Verify GatewayClass

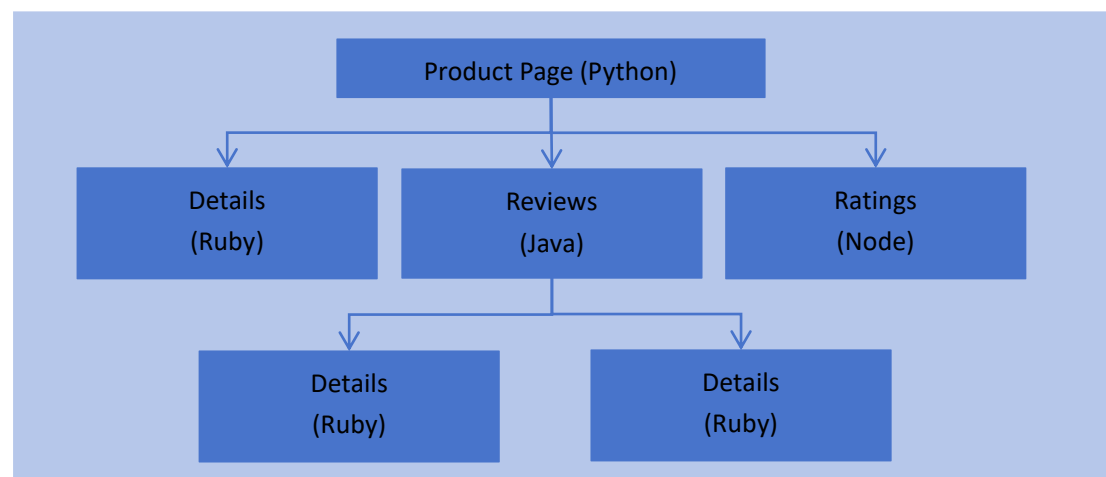
Verify the GatewayClass was created successfully.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get gatewayclass
NAME      AGE
istio     14s
```

This step bridges the Kubernetes Gateway API with Istio. The GatewayClass named istio tells the cluster that Istio's controller (istio.io/gateway-controller) is responsible for managing Gateways of this class.

5) Step 5: Deploy Sample Application

Bookinfo Application Architecture:



Deploy the Bookinfo sample application, verify the Sidecar auto-injection feature, and ensure the application can communicate normally within the cluster.

(1) Prepare Images

Manually pull one of the sample images for the Bookinfo application. (To avoid failures during deployment due to image pull issues, a sample image was pulled in advance.)

```
PS C:\Users\WangShiyu\istio-1.28.0> docker pull docker.lms.run/istio/examples-bookinfo-ratings-v1:1.20.2
1.20.2: Pulling from istio/examples-bookinfo-ratings-v1
a8e10dba16ea: Pull complete
9de058851c23: Pull complete
6e4d260b7fd8: Pull complete
6da780fd0691: Pull complete
599ac499f931: Pull complete
06d8255259bf: Pull complete
4f4fb700ef54: Pull complete
e1caac4eb9d2: Pull complete
ae9d27cb4861: Pull complete
64e9e5ee6974: Pull complete
7eec548dbdbd: Download complete
Digest: sha256:5cb3947d8967a887d9f1eeadda4ae06f5bfa29342b0155ae4a16413c6d1d6cfe
Status: Downloaded newer image for docker.lms.run/istio/examples-bookinfo-ratings-v1:1.20.2
docker.lms.run/istio/examples-bookinfo-ratings-v1:1.20.2
```

(2) Modify the Deployment File

Modify the official deployment file, replacing the image address with a domestic mirror source and unifying the version.

```
PS C:\Users\WangShiyu\istio-1.28.0> Get-Content samples/bookinfo/platform/kube/bookinfo.yaml | ForEach-Object { $_ -replace 'docker.io/istio/examples-bookinfo', 'docker.lms.run/istio/examples-bookinfo' -replace '1.20.3', '1.20.2' } | Set-Content bookinfo-custom.yaml
```

(3) Apply the Deployment File

Apply the modified deployment file.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f bookinfo-custom.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
```

Successfully created 4 microservices, including details, ratings, reviews, productpage, along with their corresponding ServiceAccounts and Deployments. The reviews service has three versions: v1, v2, and v3.

(4) Check Service and Pod Status

View all services

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get services
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
details   ClusterIP   10.101.58.79  <none>       9080/TCP   3m10s
kubernetes ClusterIP   10.96.0.1     <none>       443/TCP    30h
productpage ClusterIP   10.107.204.156 <none>      9080/TCP   3m10s
ratings   ClusterIP   10.110.44.99  <none>       9080/TCP   3m10s
reviews   ClusterIP   10.106.27.241 <none>       9080/TCP   3m10s
```

View all Pods

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-6d999b86f-h7pnl         2/2     Running   0           5m22s
productpage-v1-6fd84d55f6-ntgj5    2/2     Running   0           5m22s
ratings-v1-784cc59f6f-dt4hl        2/2     Running   0           5m22s
reviews-v1-59484bb9d5-2mhms        2/2     Running   0           5m22s
reviews-v2-8965c56b7-m655h         2/2     Running   0           5m22s
reviews-v3-65668586c7-vtflf        2/2     Running   0           5m22s
```

Observing the READY column of the Pods, all Pods show 2/2. This is crucial, as it indicates that each Pod is successfully running two containers:

- The application container: e.g., details, ratings, etc.
- The Envoy Sidecar container: Automatically injected by Istio.

Pod Status Analysis:

Pod Name	READY Status	Containers Inside (2/2)	Status
details-v1	2/2	1. details (App) 2. istio-proxy (Envoy)	Running
productpage-v1	2/2	1. productpage (App) 2. istio-proxy (Envoy)	Running
ratings-v1	2/2	1. ratings (App) 2. istio-proxy (Envoy)	Running
reviews-v1	2/2	1. reviews (App) 2. istio-proxy (Envoy)	Running
reviews-v2	2/2	1. reviews (App) 2. istio-proxy (Envoy)	Running
reviews-v3	2/2	1. reviews (App) 2. istio-proxy (Envoy)	Running

This proves that the Sidecar auto-injection feature has been successfully enabled.

(5) Verify Application Internal Connectivity

Execute a curl command from within the ratings container in the cluster to access the productpage service.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl exec (kubectl get pod -l app=ratings -o jsonpath='{.items[0].metadata.name}') -c ratings -- curl -sS productpage:9080/productpage | Select-String "<title>.*</title>"
<title>Simple Bookstore App</title>
```

The productpage service was successfully accessed from the ratings service within the cluster, and the page title was retrieved. This proves that microservices can communicate with each other within the cluster via Kubernetes' DNS service discovery, and that the Envoy Sidecar has been successfully injected and is handling traffic.

6) Step 6: Open Application to Outside Traffic

Configure an Istio Gateway and VirtualService to expose the Bookinfo application deployed in the cluster to external users.

(1) Create Gateway and VirtualService

Apply the official Gateway and VirtualService configuration file.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway created
virtualservice.networking.istio.io/bookinfo created
```

- Gateway: Defines an ingress gateway named bookinfo-gateway, listening for HTTP traffic on

port 80.

- VirtualService: Defines a routing rule named bookinfo, which routes all HTTP traffic destined for any host to port 9080 of the productpage service.

(2) Verify Configuration

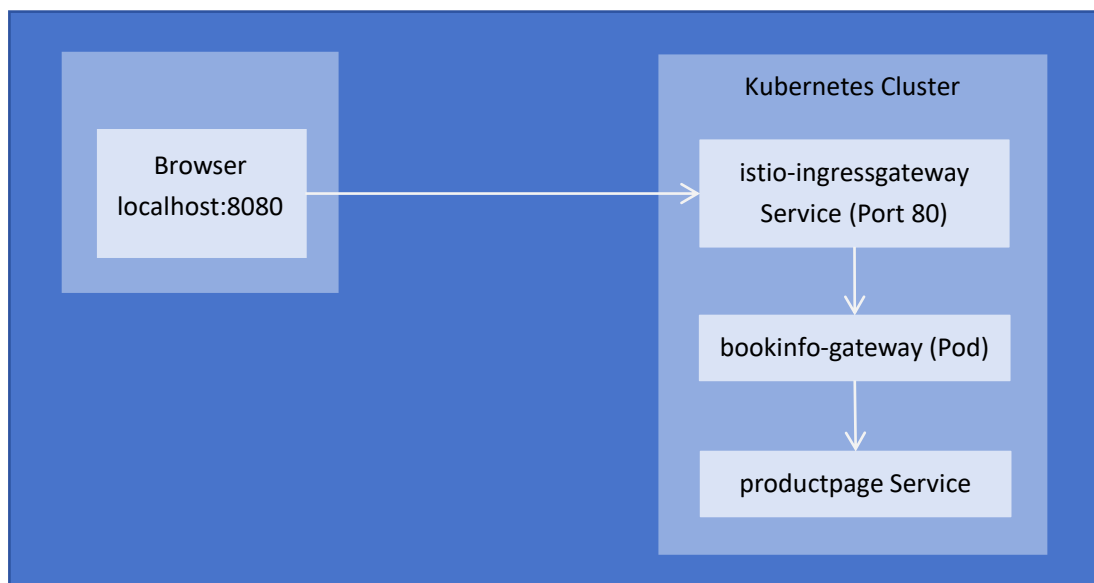
View Gateway resources, VirtualService resources and detailed Gateway configuration.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get gateway
NAME                                AGE
bookinfo-gateway                   3h36m
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice
NAME                                GATEWAYS    HOSTS    AGE
bookinfo  ["bookinfo-gateway"]  ["*"]    46m
PS C:\Users\WangShiyu\istio-1.28.0> kubectl describe gateway bookinfo
Name:          bookinfo-gateway
Namespace:     default
Labels:        <none>
Annotations:   <none>
API Version:   gateway.networking.k8s.io/v1
Kind:          Gateway
Metadata:
  Creation Timestamp:  2026-05-13T09:22:18Z
  Generation:         1
  Resource Version:    137481
  UID:                 29140abc-860e-4d03-8f2b-bbc8a6780e1a
Spec:
  Gateway Class Name:  istio
  Listeners:
    Allowed Routes:
      Namespaces:
        From: Same
      Name:      http
      Port:      80
      Protocol:  HTTP
  Events:         <none>
```

The Gateway and VirtualService resources were created successfully. The describe gateway output details its configuration: using the istio GatewayClass, listening on the http port (80), with the HTTP protocol.

7) Step 7: Access the Application

The following diagram illustrates how kubectl port-forward bridges local traffic to the Kubernetes cluster:



Access the Bookinfo application deployed in the cluster from a local browser using port forwarding.

(1) Establish Port Forwarding

Forward port 80 of the istio-ingressgateway service in the istio-system namespace to local port 8080.

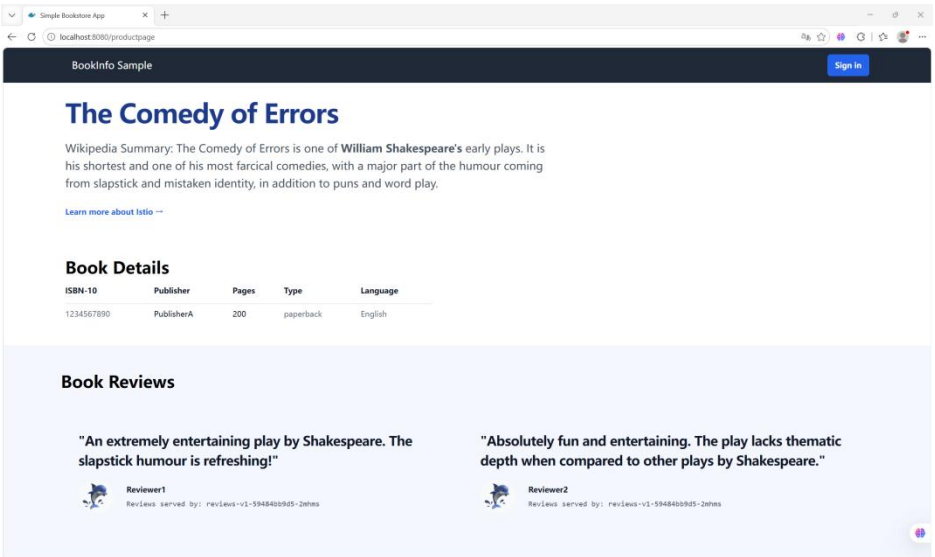
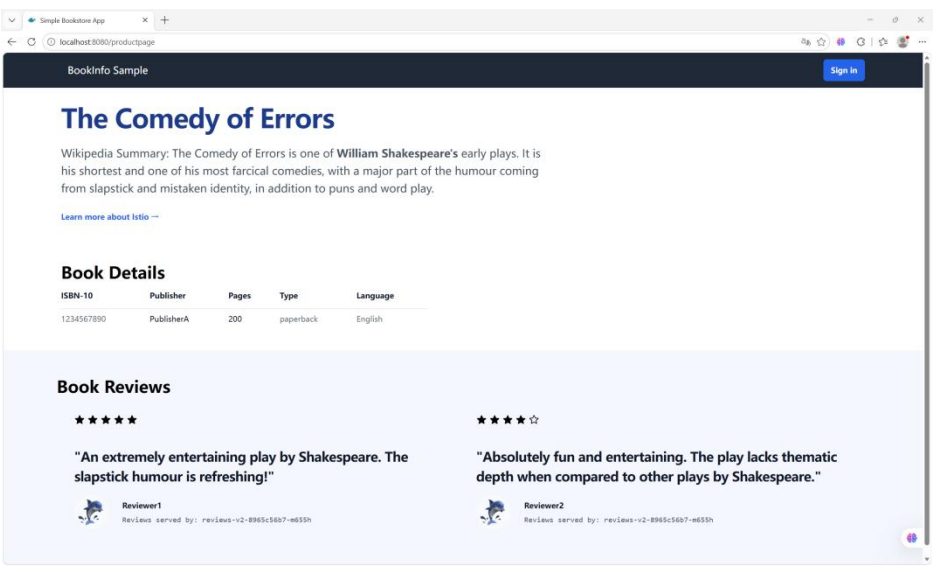
```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl port-forward svc/istio-ingressgateway -n istio-system 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

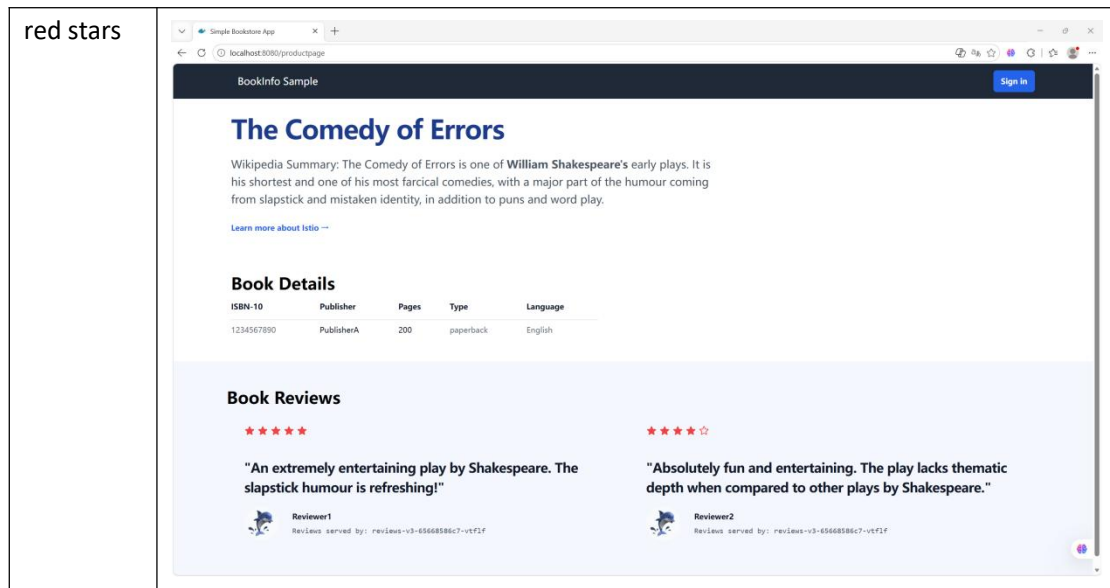
This command establishes a tunnel between the local machine and the Kubernetes cluster. All traffic sent to local localhost:8080 is forwarded to port 80 of the istio-ingressgateway service within the cluster. The traffic flow is as follows:

(2) Browser Access

Enter `http://localhost:8080/productpage` in the browser address bar.

See the main page of the Bookinfo application, displaying book details and reviews. Upon refreshing the page multiple times, you can observe the star style in the review section changing .

no stars	
black stars	



This is because traffic is being randomly distributed to the v1, v2, and v3 versions of the reviews service.

8) Step 8: Observability and Dashboard

Deploy and access the Kiali dashboard to visually observe the topology and traffic status of the service mesh.

(1) Install Kiali and Other Addons

Install Kiali, Prometheus, and other observability components.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/addons
serviceaccount/grafana created
configmap/grafana created
service/grafana created
deployment.apps/grafana created
configmap/istio-grafana-dashboards created
configmap/istio-services-grafana-dashboards created
deployment.apps/jaeger created
service/tracing created
service/zipkin created
service/jaeger-collector created
serviceaccount/kiali created
configmap/kiali created
clusterrole.rbac.authorization.k8s.io/kiali created
clusterrolebinding.rbac.authorization.k8s.io/kiali created
service/kiali created
deployment.apps/kiali created
serviceaccount/loki created
configmap/loki created
configmap/loki-runtime created
clusterrole.rbac.authorization.k8s.io/loki-clusterrole created
clusterrolebinding.rbac.authorization.k8s.io/loki-clusterrolebinding created
service/loki-memberlist created
service/loki-headless created
service/loki created
statefulset.apps/loki created
serviceaccount/prometheus created
configmap/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
service/prometheus created
deployment.apps/prometheus created
```

Kiali relies on Prometheus to collect metric data. During the experiment, the original Prometheus image failed to pull, and it was successfully deployed by using the kubectl set image command to replace its image with one from a domestic mirror source.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl set image deployment/prometheus prometheus-server=docker.m.daocloud.io/prom/
prometheus:v3.5.0 -n istio-system
deployment.apps/prometheus image updated
PS C:\Users\WangShiyu\istio-1.28.0> kubectl rollout status deployment/prometheus -n istio-system
Waiting for deployment "prometheus" rollout to finish: 0 of 1 updated replicas are available...
deployment "prometheus" successfully rolled out
```

(2) Verify Addon Status

View all Pods in the istio-system namespace.

- The Pods for kiali and prometheus show a Running status, indicating that Kiali's core functionality is available.
- grafana, jaeger, and loki are in an ImagePullBackOff state due to image pull issues, but this does not affect the core objective of this experiment (using Kiali).

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods -n istio-system
```

NAME	READY	STATUS	RESTARTS	AGE
grafana-665dcbcb9b-9qqr9	0/1	ImagePullBackOff	0	6m13s
istio-egressgateway-5484c95fdb-4dbdr	1/1	Running	0	43h
istio-ingressgateway-74c878c76-82gjr	1/1	Running	0	43h
istiod-599bd565bd-nhg8g	1/1	Running	0	43h
jaeger-75846c549c-2vv9s	0/1	ImagePullBackOff	0	6m13s
kiali-55d986fccf-q7hr5	1/1	Running	0	6m13s
loki-0	0/2	ImagePullBackOff	0	6m12s
prometheus-788d9f4dbd-w7njr	2/2	Running	0	41s

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get svc -n istio-system
```

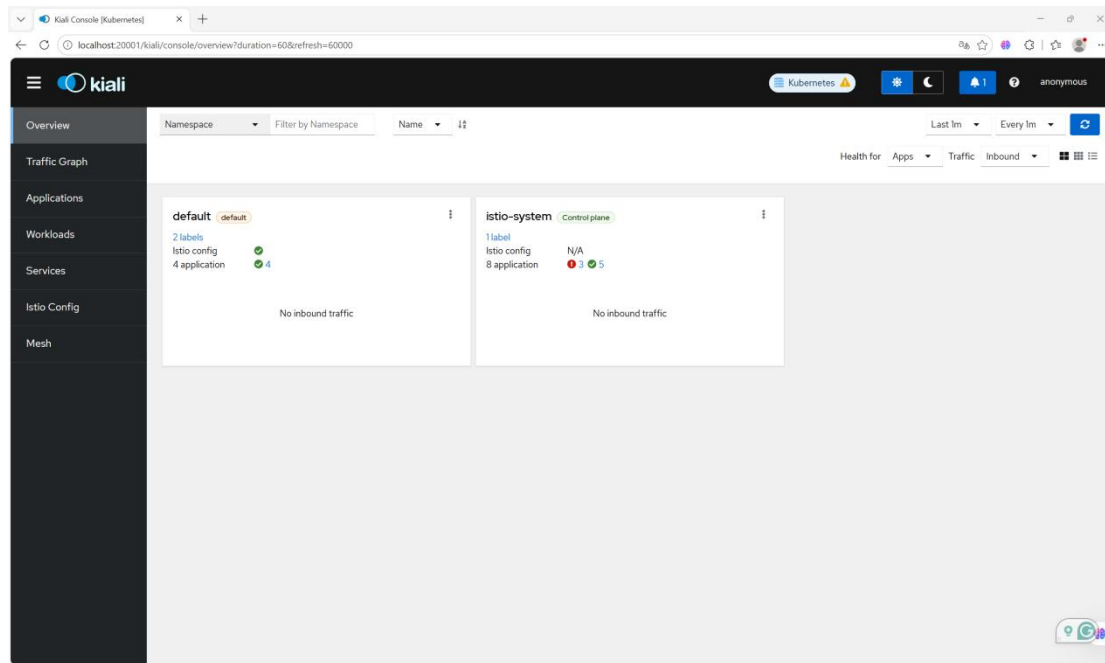
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
grafana	ClusterIP	10.108.202.142	<none>	3000/TCP
istio-egressgateway	ClusterIP	10.108.117.29	<none>	80/TCP,443/TCP
istio-ingressgateway	LoadBalancer	10.106.227.217	<pending>	15021:30724/TCP,80:31433/TCP,443:31298/TCP,31400:31389/TCP,15443:30986/TCP
istiod	ClusterIP	10.107.138.188	<none>	15010/TCP,15012/TCP,443/TCP,15014/TCP
istiod-revision-tag-default	ClusterIP	10.104.63.12	<none>	15010/TCP,15012/TCP,443/TCP,15014/TCP
jaeger-collector	ClusterIP	10.101.38.89	<none>	14268/TCP,14250/TCP,9411/TCP,4317/TCP,4318/TCP
kiali	ClusterIP	10.98.199.250	<none>	20001/TCP,9090/TCP
loki	ClusterIP	10.98.190.127	<none>	3100/TCP,9095/TCP
loki-headless	ClusterIP	None	<none>	3100/TCP
loki-memberlist	ClusterIP	None	<none>	7946/TCP
prometheus	ClusterIP	10.102.166.99	<none>	9090/TCP
tracing	ClusterIP	10.111.248.158	<none>	80/TCP,16685/TCP
zipkin	ClusterIP	10.102.230.14	<none>	9411/TCP

(3) Launch the Kiali Dashboard

Launch the Kiali dashboard in the first PowerShell window.

This command automatically opens the Kiali dashboard in the default browser (default address <http://localhost:20001/kiali>)

```
PS C:\Users\WangShiyu\istio-1.28.0> istioctl dashboard kiali
http://localhost:20001/kiali
```



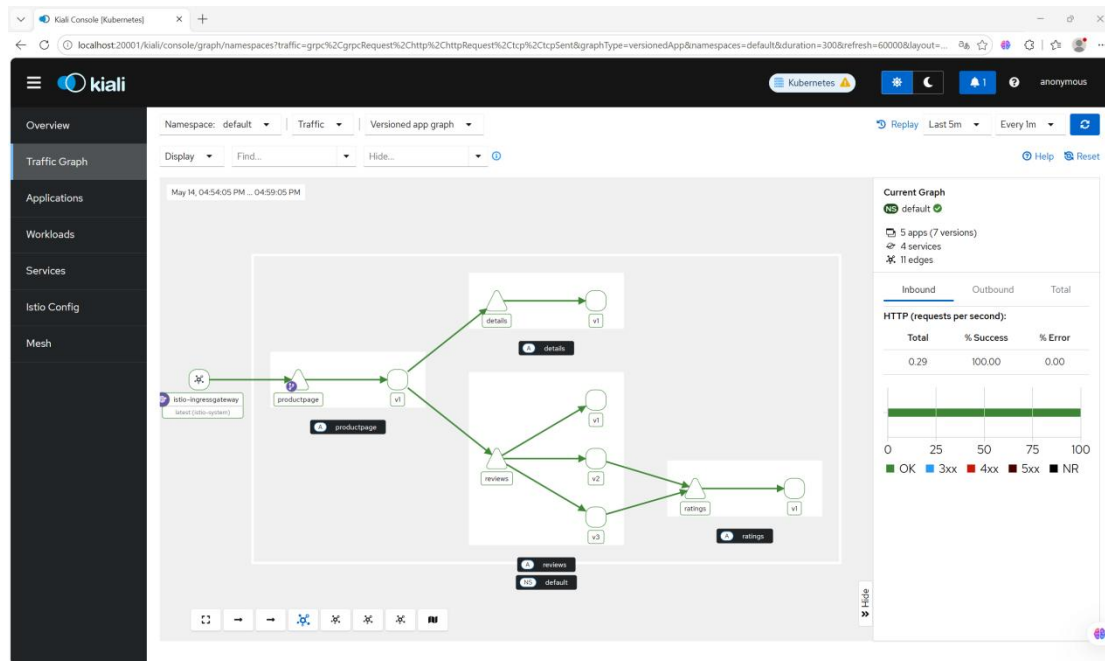
(4) Generate Traffic for Observation

In another PowerShell window, loop to send HTTP requests

```
PS C:\Users\WangShiyu> for ($i=1; $i -le 100; $i++) {
>> curl.exe -s http://localhost:8080/productpage > $null
>> Write-Host "Request $i completed"
>> Start-Sleep -Milliseconds 500
>> }
Request 1 completed
Request 2 completed
Request 3 completed
Request 4 completed
Request 5 completed
Request 6 completed
Request 7 completed
Request 8 completed
Request 9 completed
Request 10 completed
.....
Request 90 completed
Request 91 completed
Request 92 completed
Request 93 completed
Request 94 completed
Request 95 completed
Request 96 completed
Request 97 completed
Request 98 completed
Request 99 completed
Request 100 completed
```

(5) Observation Results

- In the Kiali dashboard's left navigation bar, select Graph.
- In the Namespace dropdown, select default.
- Observe the service topology map as traffic begins to flow.



Observation Metrics:

Metric	Value	Interpretation
HTTP Requests	~100 requests	Total traffic generated by the script.
Success Rate	100%	All services are healthy.
Traffic Distribution	~33% each	Even distribution across reviews v1, v2, v3.

Kiali Service Topology Interpretation:

- **Service Nodes:** Each box represents a service, such as productpage, details, reviews, etc.
- **Connection Lines and Arrows:** Show the invocation relationships and traffic direction between services. For example, istio-ingressgateway points to productpage, which in turn points to details and reviews.
- **Traffic Information:** Key metrics like traffic percentages, request success rates, and average response times are annotated on the connection lines, providing an intuitive insight into service health and performance.

3. Conclusion

This experiment successfully completed the introductory task of Istio Service Mesh, achieving the installation and deployment of Istio, the deployment of a microservice application, the exposure of external traffic, and the visual observation of the service mesh.

1) Key Takeaways

(1) **Combination of Theory and Practice:** Through hands-on operation, abstract concepts like Istio's control plane, data plane, Gateway, and VirtualService were transformed into concrete

Kubernetes resources and running states.

(2) **Problem-Solving Skills in Constrained Environments:** Mastered practical techniques for solving image pull issues in non-ideal network environments by modifying configurations and replacing image sources.

(3) **End-to-End Process Understanding:** Successfully completed the full chain from infrastructure preparation to final application accessibility, gaining a deep understanding of how the Service Mesh "non-intrusively" integrates with and manages microservice traffic.

(4) **Appreciation for the Value of Observability:** Through the Kiali dashboard, intuitively experienced the powerful observability capabilities brought by the service mesh and understood its significant value for microservice operations.

2) Problems Encountered and Solutions

Problem Description	Cause Analysis	Solution
Slow/Failed Istio Image Pulls	Unstable access to Docker Hub from domestic networks	Created a custom IstioOperator configuration file and set the hub parameter to the domestic mirror source <code>docker.1ms.run/istio</code> .
Failed Bookinfo App Image Pulls	Same as above	Modified the <code>bookinfo.yaml</code> file, replacing all image addresses with those starting with <code>docker.1ms.run/istio</code> .
Prometheus Image Pull Failure	Addon images also come from Docker Hub	Used the <code>kubect</code> l set image command to replace the Prometheus image with <code>docker.m.daocloud.io/prom/prometheus:v3.5.0</code> .
Grafana, Jaeger, Loki Image Pull Failures	Image source issue	As Kiali and Prometheus were running successfully, satisfying the core objective of the experiment, no further action was taken. In a production environment, all image source issues would need to be resolved.